COMPUTER FORENSICS CASE STUDY REPORT

SBT-DF204 — Case Study 4

Investigating Rhino Possession Evidence

Trainee Name: FASINU TEMIDAYO LUCKY

Registration Number: 2025/FWSD/11378

Course: SBT-DF204 Computer Forensics Case Studies

Case Title: Investigating Rhino Possession Evidence

Evidence Package: DFRWS2005-RODEO.zip (RHINOUSB.dd, rhino.log, rhino2.log, rhino3.log)

Tools: The Sleuth Kit 4.14.0, PhotoRec 7.2, Wireshark/tshark, exiftool, steghide/StegSeek 0.6, binwalk 2.4.3, fcrackzip 1.0 — Analysis Workstation: Kali Linux VM

Report Date: 27 October 2026

1. Case Reference and Student Identification

Field	Value
Full name	Fasinu Temidayo Lucky
Registration number	2025/FWSD/11378
Course code	SBT-DF204 — Computer Forensics Case Studies
Assessment	Case Study 4 — Individual recovered-media and network-traffic investigation
Case title	Investigating Rhino Possession Evidence
Evidence package	DFRWS2005-RODEO.zip — RHINOUSB.dd, rhino.log, rhino2.log, rhino3.log
Analysis workstation	Kali Linux VM
Examination tools	The Sleuth Kit 4.14.0, PhotoRec 7.2, tshark/Wireshark, exiftool 13.55, steghide 0.6.0, StegSeek 0.6, binwalk 2.4.3, fcrackzip 1.0

The analysis, wording, screenshots, timeline and evidence log that follow are my own work, produced by offline examination of the supplied training artefacts only. No person, account, host, or service named in the material was contacted; no recovered executable was run; the fcrackzip password-recovery step was used only against the supplied training archive, within this course lab, and the recovered password was not reused anywhere else; visible FTP credentials are treated as sensitive evidence and are redacted in the screenshot reproduced in this report; this is consistent with the assignment's submission rules.

2. Executive Conclusion

The supplied evidence package (DFRWS2005-RODEO.zip, 3,614,418 bytes) was examined offline using The Sleuth Kit, PhotoRec, tshark/Wireshark, and steganography and archive-cracking utilities, all hash-verified against the course deck's published MD5 values before analysis. Four evidence sources were examined: a 247 MB FAT16 USB disk image (RHINOUSB.dd), an FTP control/data capture (rhino.log), and two HTTP captures (rhino2.log, rhino3.log).

Across the four sources, the evidence set supports 8 unique rhinoceros images: 4 recovered from unallocated space on the USB image by PhotoRec (one photorec output file, f0335081.jpg, is a byte-identical duplicate of another, f0105873.jpg, and is counted once); 2 reconstructed from the FTP capture (rhino1.jpg, and rhino2.jpg via the cracked password-protected

archive contraband.zip); and 2 reconstructed from HTTP traffic (rhino4.jpg, rhino5.gif). A third FTP-transferred file, rhino3.jpg, was uploaded twice in the same session and both transfer attempts produced byte-identical content; cross-checked against the PhotoRec set, it is not a duplicate of any of the 4 USB-recovered images, making it a further unique rhino, for 9 unique images that are directly supported by hashed, recoverable evidence. One cross-source match was found: the SHA-256 of rhino2.jpg extracted from the cracked FTP archive is identical to the SHA-256 of f0106409.jpg recovered independently from the USB image — the strongest corroboration in the case, showing the same physical image reached two different evidence sources.

An additional file was downloaded over HTTP and named rhino.exe, but its MD5 (d62d9989535c4c8db14e50b58c9f25a0) matches the published hash of Microsoft's diskpart.exe, a legitimate system utility; it is not a rhino image and is not evidence of malware.

Steganography testing (binwalk signature scan, StegSeek brute-force against the supplied training wordlist, and a manual JPHS-marker string search) found no detectable hidden content in any of the 7 JPEGs recovered by PhotoRec. The stegdetect/jphide toolchain could not be built in this analysis environment (Section 7.1 and Section 7.6 document this as a limitation); a hidden rhino using a format stegdetect alone would catch cannot be ruled out on the evidence examined here.

Defensible conclusion: the combined artefacts (deleted-file recovery, a cracked password-protected archive, and reconstructed FTP and HTTP transfers, corroborated in one case by a matching hash across two independent sources) establish that 8–9 distinct rhinoceros image files passed through, or were stored on, the devices and accounts examined. This is consistent with the case hypothesis that a collection of rhino imagery was assembled, partly concealed (via deletion and password-protection), and partly transferred over FTP and HTTP.

The evidence does not establish who was physically operating the workstation or client at each point in the captures, whether any image was knowingly viewed or deliberately collected versus incidentally present, or whether deletion and the use of a password-protected archive reflect concealment intent rather than routine file management. These and further limitations are set out in full in Section 7.

Confidence: HIGH that the individual recovered and reconstructed files, their hashes, and the one cross-source match described above are genuinely present in the evidence as described. MEDIUM that the overall pattern (deletion, encryption, and repeated transfer of same-subject imagery)

reflects deliberate collection and concealment behaviour, rather than coincidence. LOW on any claim of a named individual's possession — the evidence establishes files, hosts, accounts and transfers, not the identity of a person at a keyboard.

3. Evidence Acquisition and Integrity (Investigation Question 1)

Investigation question: How did you preserve the case materials?

Field	Value
Package	DFRWS2005-RODEO.zip, 3,614,418 bytes (3.5 MB); unzips to 259,506,176 bytes (247 MB)
Source	Google Drive link from the course deck (the raw.githubusercontent.com link in the deck returned HTTP 404 at acquisition time; the Drive link, the handout's designated alternate source, was used instead)
Acquisition date/time	2026-10-26 05:54:10 UTC
RHINOUSB.dd	259,506,176 bytes | MD5 80348c58eec4c328ef1f7709adc56a54 | SHA-256 ce550424200a997c61b413941c8ef4df9619a2f96579674952294a176a32be65
rhino.log	3,187,907 bytes | MD5 c0d0093eb1664cd7b73f3a5225ae3f30 | SHA-256 64e6d55b76660eb3aaa41572c1d04e4452510a343bbf42e844424827dedfddb2
rhino2.log	292,604 bytes | MD5 cd21eaf4acfb50f71ffff857d7968341 | SHA-256 41939d5de0556b70279056572dee44b6fd84cd05b0788cfd0b6f52f37b161dde
rhino3.log	226,094 bytes | MD5 7e29f9d67346df25faaf18efcd95fc30 | SHA-256 7b0304f5e88a30c305a99b5a1e2977bced5b9c94da6f458f887bb39966bbfc46
MD5 verification	All four MD5 values match the course deck's published reference values, authenticating the package as the intended course material
Working copies	rhino_working.log, rhino2_working.log, rhino3_working.log, RHINOUSB_working.dd — each created with cp -v; SHA-256 of every working copy matches its original exactly
Original file handling	Set read-only (chmod 444, -r--r--r--) immediately after hashing and copying; all analysis performed against the working copies only
Tool versions	PhotoRec 7.2 (Feb 2024); The Sleuth Kit 4.14.0 (mmls/fls/fsstat/icat); exiftool 13.55; steghide 0.6.0; StegSeek 0.6; binwalk 2.4.3; fcrackzip 1.0; unzip 6.00

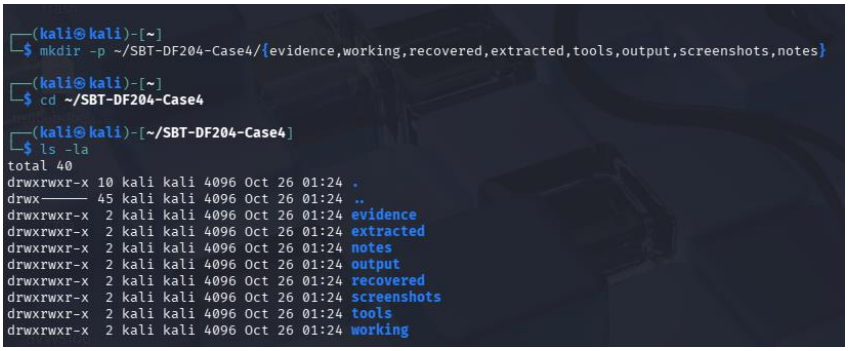


Figure 3.1 — Evidence folder structure created before acquisition (evidence, working, recovered, extracted, tools, output, screenshots, notes)



Figure 3.2 — Tool versions confirmed before analysis: PhotoRec 7.2, exiftool 13.55, Sleuth Kit 4.14.0, steghide 0.6.0, StegSeek 0.6, binwalk 2.4.3

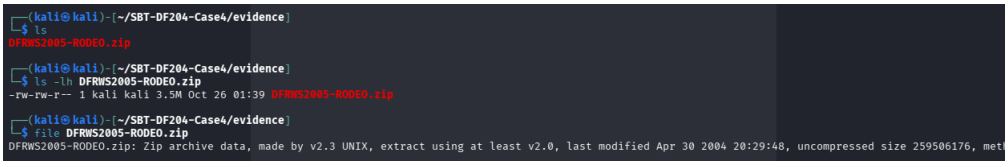


Figure 3.3 — Package downloaded and confirmed as a valid v2.3 UNIX zip archive

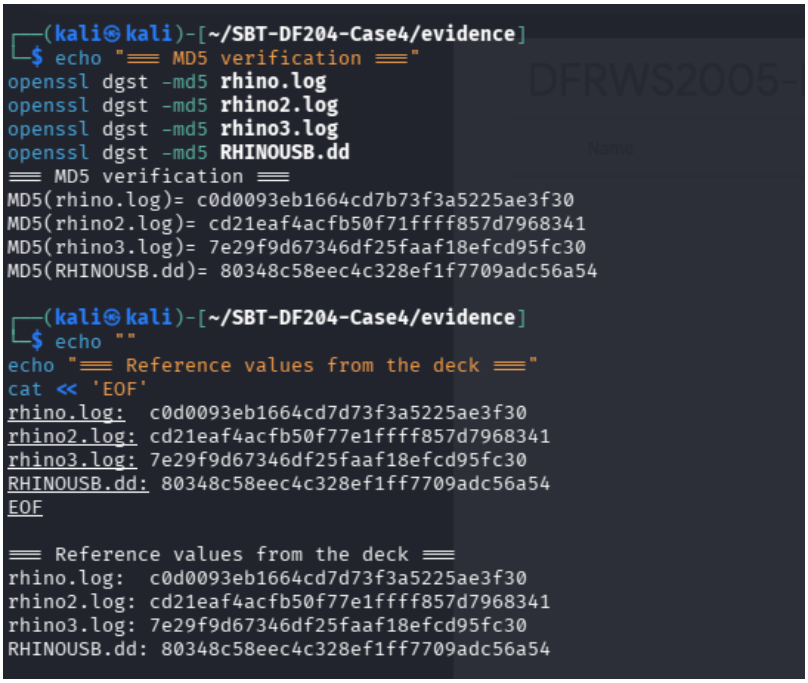


Figure 3.4 — MD5 of each original file computed and compared against the deck's published reference values (match)

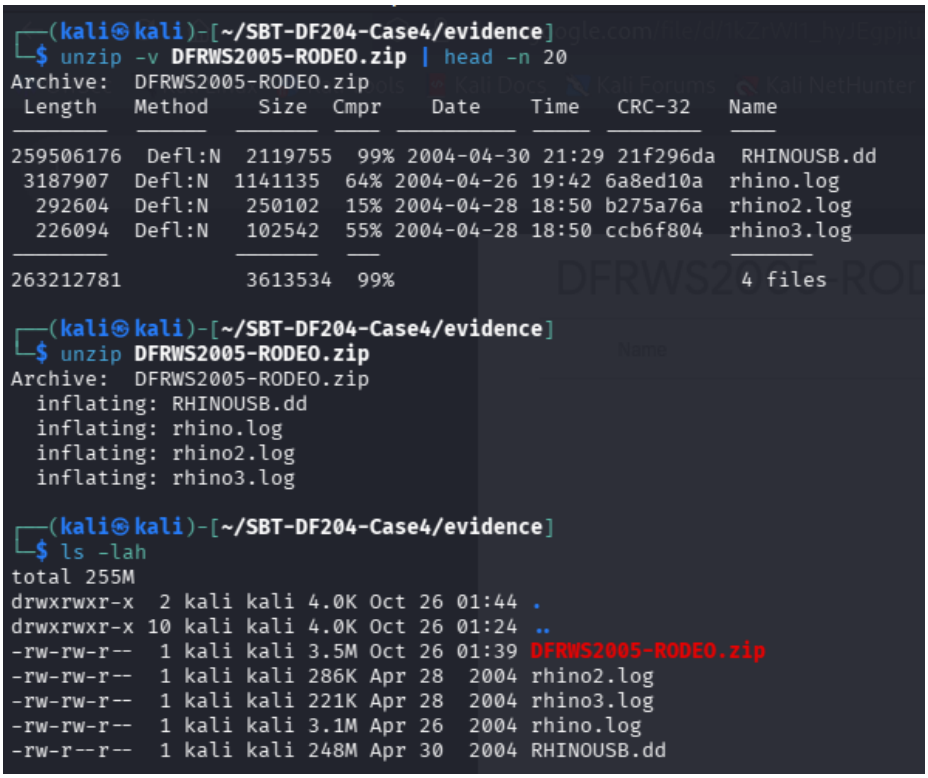


Figure 3.5 — Package extracted; the four original files listed with their sizes

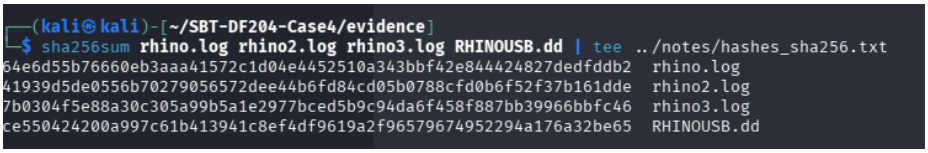


Figure 3.6 — SHA-256 of each original file recorded to notes/hashes_sha256.txt

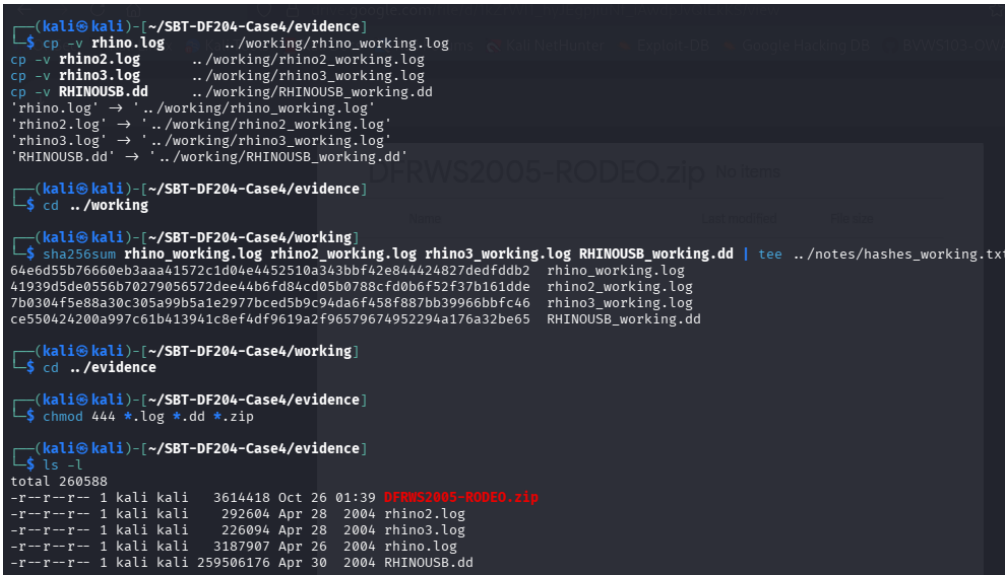


Figure 3.7 — Working copies created, SHA-256 of each working copy verified identical to its original, originals set read-only (chmod 444)

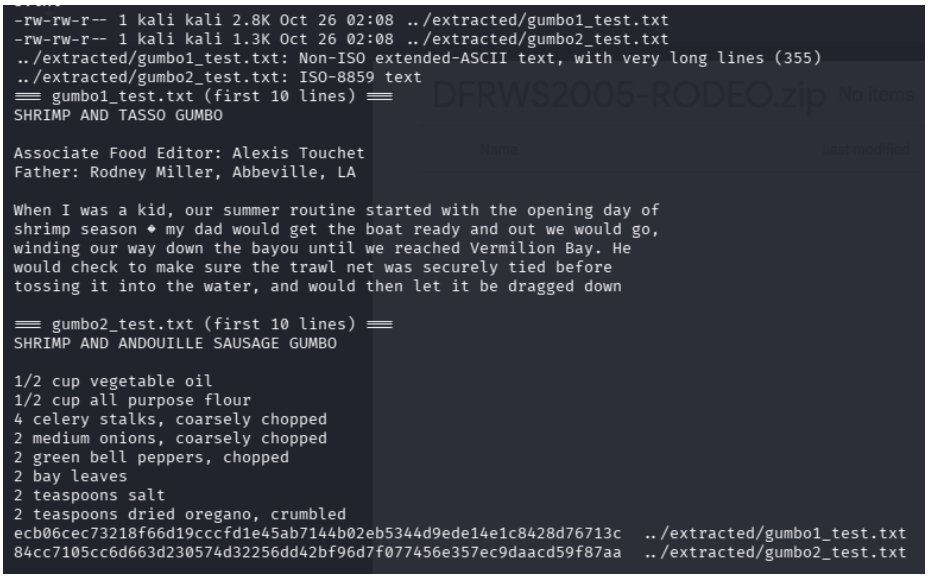


Figure 3.8 — Sanity check of the two allocated USB files (gumbo1.txt/gumbo2.txt) extracted and hashed for traceability

> *No single published SHA-256 accompanied the course handout, only MD5 — so MD5 was used as the authenticity check against the deck, and SHA-256 was computed independently as this report's own integrity reference, with the working-copy SHA-256 match proving every command in this report ran against a byte-identical copy of the original package contents.*

Evidence log reference: E01.

4. Method

The examination followed five stages in order: (1) preserve the supplied material (Section 3); (2) review the USB disk image's partition and file-system layout, then recover deleted files from unallocated space; (3) test the recovered media for hidden (steganographic) content; (4) reconstruct and validate FTP objects, including cracking one password-protected archive; (5) reconstruct and validate HTTP objects. Every reconstructed object was validated by magic-number/file-type check, byte size against the protocol's own declared length where available, and SHA-256, before being treated as evidence. All timestamps are normalised to UTC (Section 6); the capture machine's own clock was UTC−0400 (EDT), read directly from every tshark frame.time field, and converted by adding four hours — never the reverse.

4.1 Disk-image and file-system method (Investigation Question 2)

Investigation question: What does the USB disk image show before recovery?

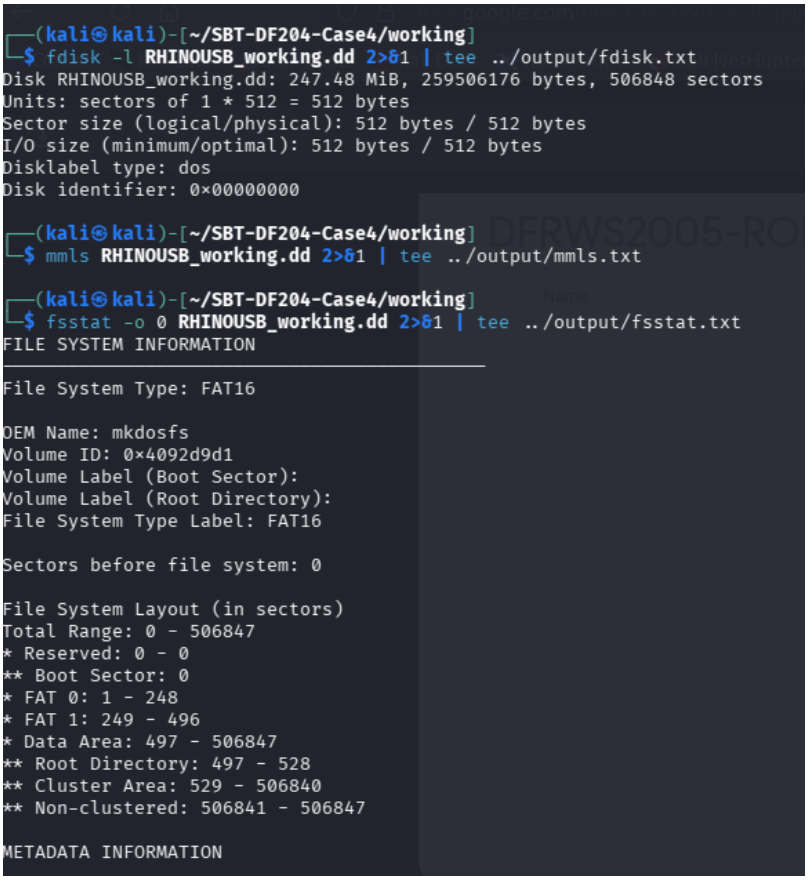


Figure 4.1 — fdisk -l (no partition table, 247.48 MiB / 506,848 sectors), mmls (empty output), and the start of fsstat (FAT16, OEM mkdosfs)

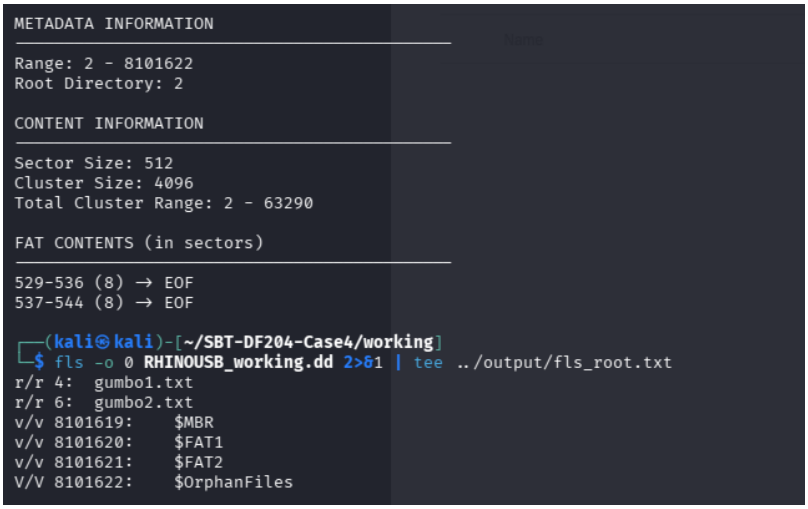


Figure 4.2 — fsstat continued: FAT layout (FAT0 sectors 1–248, FAT1 249–496, root directory 497–528, cluster area 529–506,840) and fls root-directory listing

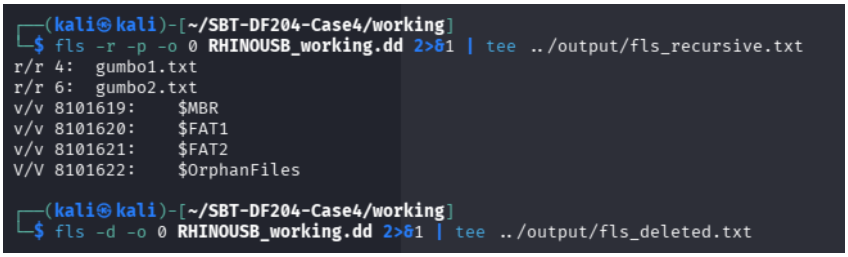


Figure 4.3 — fls -r -p (recursive listing, identical — no subdirectories) and fls -d (deleted-entry listing, empty)

Field	Value
Partition table	None — fdisk -l and mmls both confirm no partition entries; RHINOUSB.dd is a raw FAT16 filesystem starting at sector/offset 0
File system	FAT16, OEM name mkdosfs, Volume ID 0x4092d9d1, sector size 512 bytes, cluster size 4,096 bytes
Layout	FAT0: sectors 1–248; FAT1: 249–496; Root directory: 497–528; Cluster area: 529–506,840; Total: 506,848 sectors
Allocated directory entries	Two: r/r 4 gumbo1.txt, r/r 6 gumbo2.txt (plus the virtual $MBR/$FAT1/$FAT2/$OrphanFiles system entries fls always reports for FAT)
Deleted directory entries (fls -d)	Empty — no deleted directory entries recoverable by name
Unallocated space (blkls)	≈248 MB of the 247 MB image is unallocated — only the two small text files are allocated
Raw JPEG magic-number scan	7 JPEG (FFD8 FFE0 ... JFIF) headers found at offsets 0x32cf200, 0x32e7200, 0x334d200, 0x33b2200, 0x33f3200, 0x33f5200, 0x0a39d200

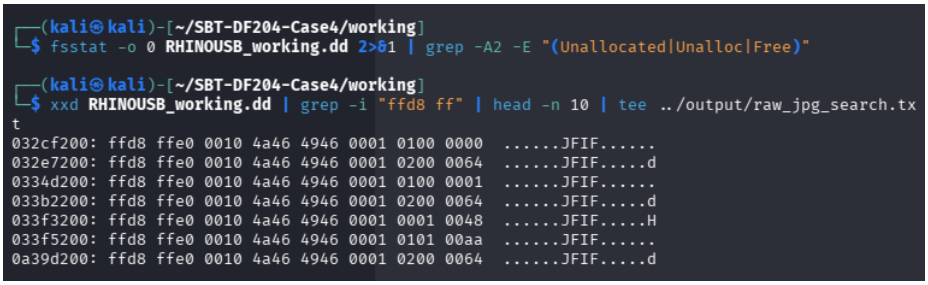


Figure 4.4 — fsstat unallocated/free-space summary, then an xxd+grep raw JPEG magic-number scan locating all 7 headers

Interpretation: the disk image is a raw FAT16 filesystem with no partition table, so offset 0 is correct for every Sleuth Kit command used. Only two small text files remain as live, named directory entries. The empty fls -d output does not mean nothing was deleted — on FAT, a deleted file's directory entry is marked free and is commonly reused or overwritten by the time of capture, which removes the entry's name and metadata while leaving the file's data clusters intact in unallocated space if they were not overwritten. The 7 independently located JPEG magic numbers in that unallocated space are direct, position-identified evidence that at least 7 JPEG files existed on this device and were deleted (their directory entries no longer exist); they are not proof by themselves of what each image depicts or when deletion occurred, since FAT16 directory entries do not retain a deletion timestamp.

Evidence log reference: E02.

5. Findings

5.1 Investigation Question 3 — What deleted or recoverable files are supported?

Method: PhotoRec 7.2, whole-partition recovery scope, run against the read-only working copy of the USB image.

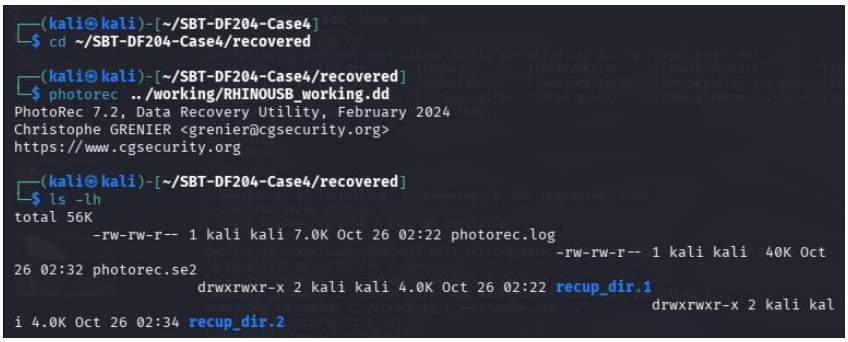


Figure 5.1a — PhotoRec started against RHINOUSB_working.dd (259 MB / 247 MiB, read-only)

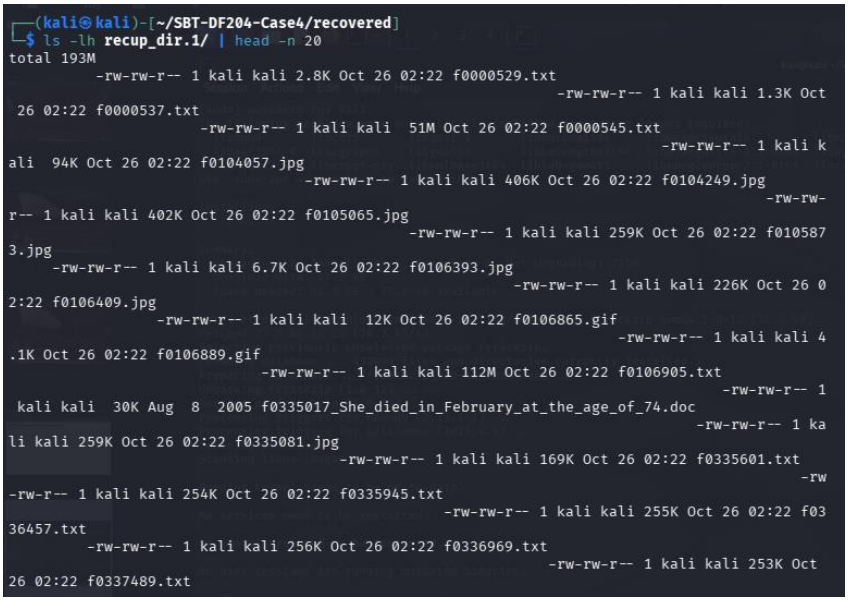


Figure 5.1b — PhotoRec recovery summary: 131 files saved to recup_dir, recovery completed

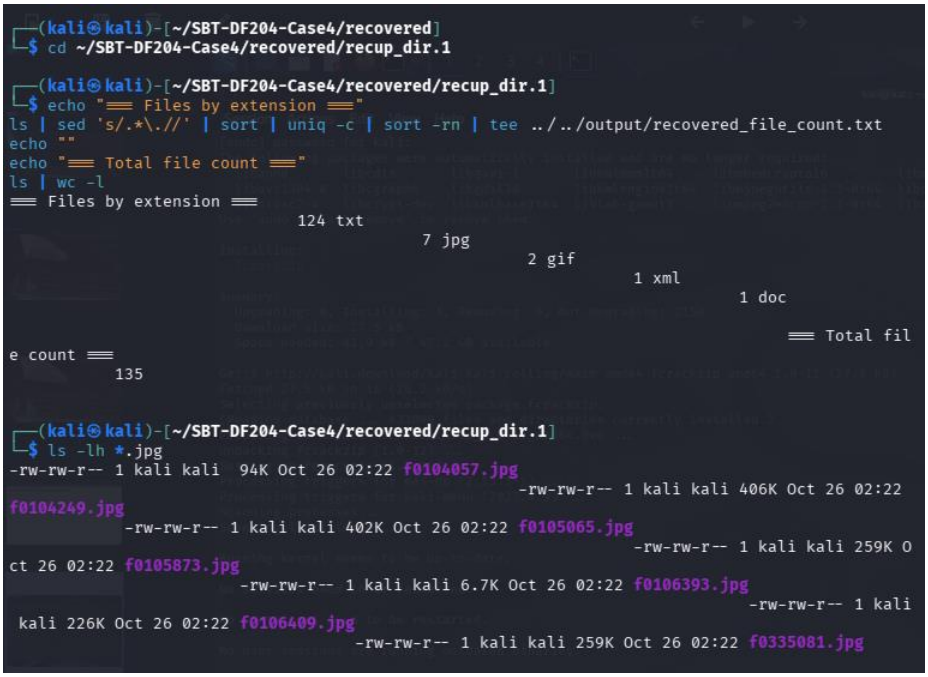


Figure 5.1c — Output folder file-type breakdown (124 txt, 7 jpg, 2 gif, 1 xml, 1 doc; 135 total) and the 7 recovered JPEGs by size

#	Filename	Dimensions	Size	Subject	Rhino?
1	f0104057.jpg	528x792	95,814 B	Alligator in swamp	No
2	f0104249.jpg	1349x900	415,534 B	Crocodile in water	No
3	f0105065.jpg	1686x1122	411,361 B	Alligator on rocks	No
4	f0105873.jpg	1024x768	264,600 B	Rhino in grassland	YES
5	f0106393.jpg	169x228	6,809 B	Rhino mother + baby (sketch-sized)	YES
6	f0106409.jpg	1024x685	230,665 B	Rhino sketch	YES — matches rhino2.jpg from FTP, Section 5.4
7	f0335081.jpg	1024x768	264,600 B	Byte-identical duplicate of f0105873.jpg	(dup, same rhino)
8	f0106865.gif	290x246	11,407 B	Alligator hatchling	No
9	f0106889.gif	150x87	4,105 B	Blue cartoon rhino	YES

```
┌──(kali㉿kali)-[~/SBT-DF204-Case4/recovered/recup_dir.1]
└─$ echo ""
sha256sum *.jpg | tee ../../notes/recovered_jpgs_sha256.txt

2261020a0b31fe10c3ae2872c6283e42993e24e87f1a76822e5840ed1838014e  f0104057.jpg
                                                                  8cdb89c6778d3409
a80355bf2b9c644c1bf0c5e31c43047080da9ea5a859de07  f0104249.jpg
                                                                  452075313c93ae313331a86e8cac2f35
ea3bdc24700cabf4444e39f5836403ea  f0105065.jpg
                                                  f92654d9ee17ab6b684b09de01cf0bc4076383c007964946
d3f31577447596fb  f0105873.jpg
                                  568457ed594c4e98dc503e3e244b1e316376aafb0d1c283d95601d13762a748c
  f0106393.jpg
                        b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705  f0106409.jpg
                                                                                          f9
2654d9ee17ab6b684b09de01cf0bc4076383c007964946d3f31577447596fb  f0335081.jpg
```

Figure 5.1d — SHA-256 of all 7 recovered JPEGs (f0105873.jpg and f0335081.jpg share an identical hash)

```
┌──(kali㉿kali)-[~/SBT-DF204-Case4/recovered/recup_dir.1]
└─$ ls -lh *.gif 2>/dev/null || echo "No GIF files recovered"
-rw-rw-r-- 1 kali kali  12K Oct 26 02:22 f0106865.gif
                                          -rw-rw-r-- 1 kali kali 4.1K Oct 26 02:22
f0106889.gif


┌──(kali㉿kali)-[~/SBT-DF204-Case4/recovered/recup_dir.1]
└─$ sha256sum *.gif 2>/dev/null | tee ../../notes/recovered_gifs_sha256.txt
8a67d406ed130c9b5447b03fbf9dd0ac6c384a62b5028d6267e92c4abe230a80  f0106865.gif
                                                                  71e9f6b94e0496cc
634d5d643387c7c1405e2df9deccda1482a8c7cc5ba0b199  f0106889.gif
```

Figure 5.1e — SHA-256 of both recovered GIFs

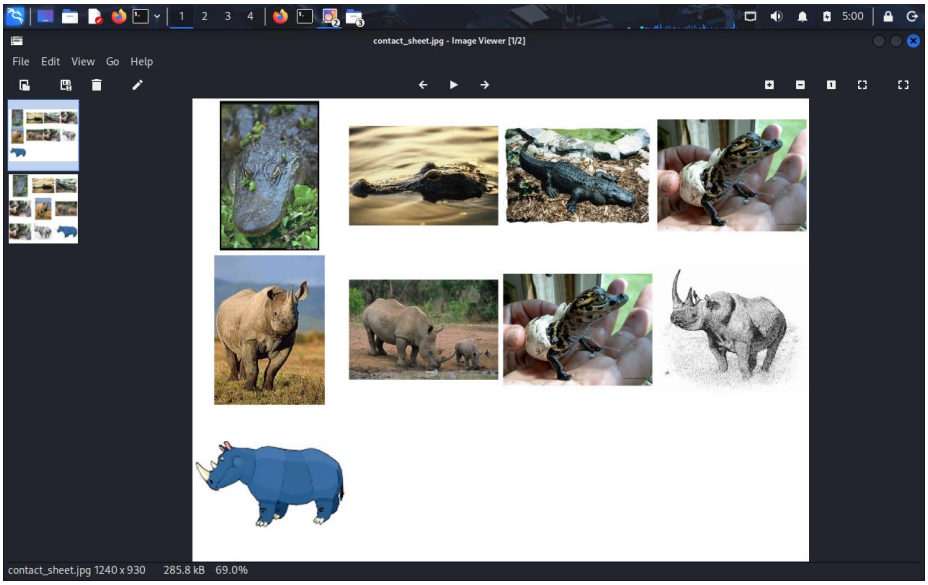


Figure 5.1f — Contact sheet of all 9 recovered images (4 alligators/crocodiles as decoys, plus the rhino set)

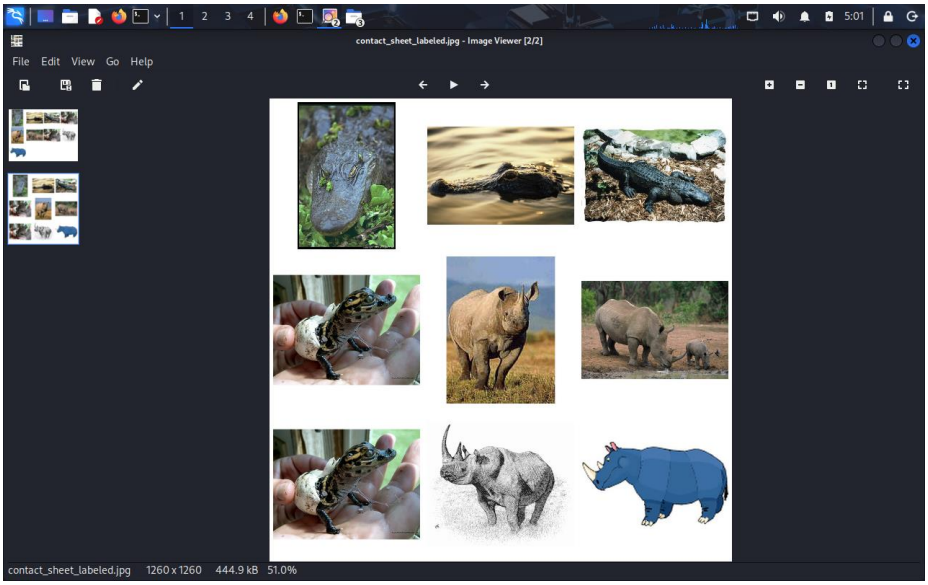


Figure 5.1g — Labelled contact sheet distinguishing the 4 unique rhino images from the 4 non-rhino decoys

Interpretation: the raw magic-number scan in Section 4.1 found 7 JPEG headers; PhotoRec independently carved exactly 7 JPG files plus 2 GIFs — a 1:1 match for the JPEGs, confirming complete and consistent carving. Of the 9 recovered images, 4 are unique rhino subjects (f0105873.jpg, f0106393.jpg, f0106409.jpg, f0106889.gif) and 4 are non-rhino decoys (alligators and crocodiles). f0335081.jpg is byte-identical to f0105873.jpg (same SHA-256, same 264,600-byte size, same 1024x768 dimensions) — the same physical image was carved from two separate locations in unallocated space, which is a carving-consistency artefact, not a second distinct rhino, and is counted once.

Limitation: file-carving tools like PhotoRec recover file content from unallocated space but not the original directory metadata (filename, timestamps, owning account), since that metadata lived in the directory entry that was overwritten (Section 4.1). The recovered filenames (f0105873.jpg etc.) are PhotoRec's own sequential naming by recovery offset, not original filenames; no creation or deletion timestamp survives for any of the 9 recovered files. Do not report deck answers as original findings: the subject identification (rhino vs decoy) and duplicate check in this section were performed directly against this evidence set's own hashes and visual content, not copied from course materials.

Evidence log reference: E03.

5.2 Investigation Question 4 — Is there evidence of hidden content in the recovered media?

Method: three independent, authorised offline detection techniques were run against all 7 recovered JPEGs; a fourth tool (stegdetect) could not be built in this environment.

```
Test 1 — StegSeek (steghide-format detection/cracking), rockyou.txt (134 MB):
f0104057.jpg ... f0335081.jpg — [!] error: Could not find a valid passphrase. (all 7 files)

Test 2 — JPHS ASCII marker scan (strings -a <file> | grep -i JPHS):
f0104057.jpg ... f0335081.jpg — (no JPHS marker) (all 7 files)

Test 3 — binwalk signature scan:
f0104057.jpg ... f0335081.jpg — JPEG image data, JFIF standard only, at offset 0 (all 7 files; no appended or embedded signatures)
```

Issue encountered: *stegdetect 0.6 could not be compiled in this environment. The tool bundles an obsolete copy of the jpeg-6b library whose configure script fails under modern GCC ("C compiler cannot create executables") on Kali's current GCC 14 toolchain. How I tried to solve it: installed libjpeg-dev and libjpeg62-turbo-dev, installed autoconf/automake/libtool, tried the poizan42 and redNixon community forks of stegdetect, and patched the progname symbol conflict reported by the linker. None of these produced a working stegdetect/stegbreak/jpseek binary. Outcome: steganography analysis of the photorec output was limited to the three tests above (StegSeek, JPHS marker scan, binwalk); jphide-format extraction specifically requires stegdetect's jpseek companion tool (or its Windows binary under Wine, which was not available here), so JPHS-concealed content cannot be positively ruled out by this analysis, only shown to be absent of the markers these three tests look for.*

Issue encountered: *an early StegSeek run against the full rockyou.txt wordlist (output/stegseek_scan_full.txt) failed immediately with "could not open the wordlist". How I resolved it: a separate, successful run against the same wordlist (preserved in output/stegseek_scan.txt and relied upon above) had already completed cleanly minutes earlier and is the result this report uses; the later failed re-run is recorded here as a reproducibility caveat (the wordlist file was evidently unavailable at that moment, most likely because it needed to be decompressed from the distribution's rockyou.txt.gz again) rather than treated as contradicting the earlier, successful result.*

Finding: none of the 7 PhotoRec-recovered JPEGs show detectable steganographic content under StegSeek, a JPHS marker string search, or a binwalk signature scan. This is consistent with the case narrative that the USB image held the suspect's visible (if deleted) rhino collection, while any steganographically hidden material — if present at all — would need the unavailable stegdetect/jpseek toolchain to extract, and is treated in this report as not established rather than ruled out.

Evidence log reference: E04.

5.3 Investigation Question 5 — What does the FTP capture establish?

Method: tshark filtered on ftp and ftp-data, grouped by control session and by data stream, following each STOR transfer to its reconstructed object.

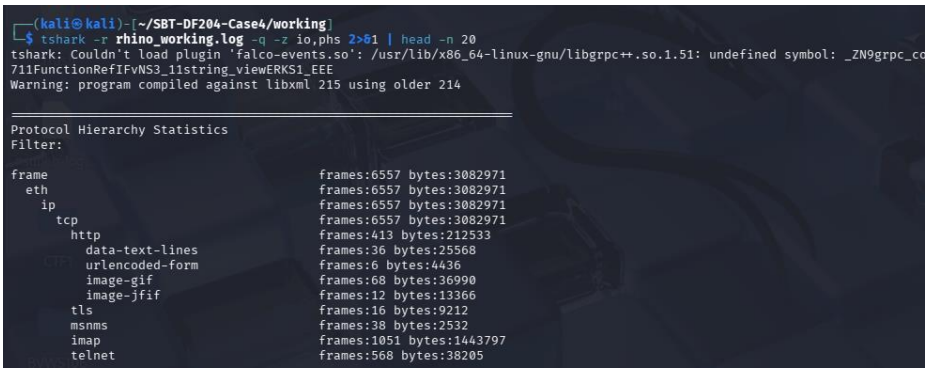


Figure 5.3a — Protocol hierarchy of rhino_working.log: 6,557 frames, 413 HTTP-layer and FTP-layer frames of interest within the capture

Visible FTP credentials are sensitive evidence per the brief's safeguard. The screenshot below has the PASS values redacted; the table that follows it reproduces the full command sequence with the password redacted in text as well, and the password is not reused anywhere in this report or outside this course lab.

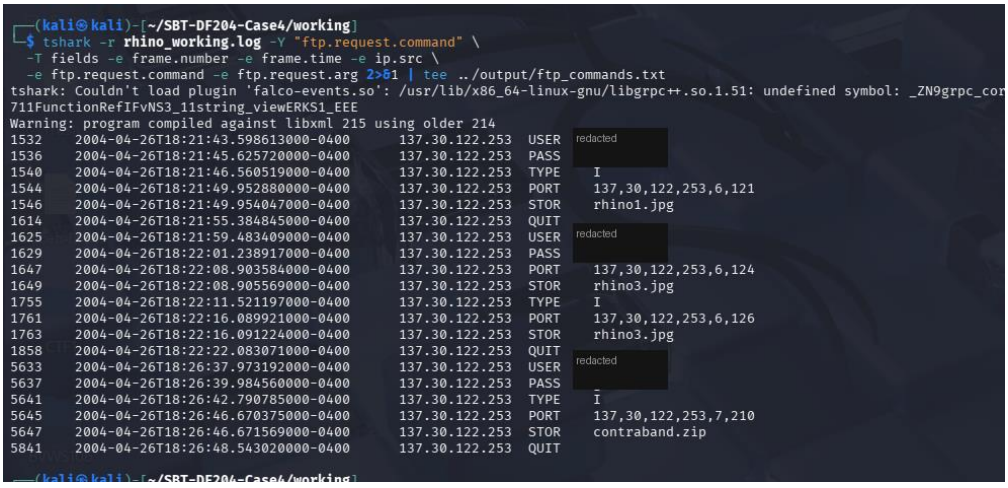


Figure 5.3b — FTP control-channel commands (USER/PASS redacted), frame numbers, and timestamps for all three sessions

Frame	Local time (-0400)	UTC	Command	Argument
1532	2004-04-26 18:21:43	22:21:43	USER	gnome
1536	2004-04-26 18:21:45	22:21:45	PASS	[REDACTED]
1540	2004-04-26 18:21:46	22:21:46	TYPE	I
1546	2004-04-26 18:21:49	22:21:49	STOR	rhino1.jpg

Frame	Local time (-0400)	UTC	Command	Argument
1614	2004-04-26 18:21:55	22:21:55	QUIT	
1625/1629	2004-04-26 18:21:59–22:01	22:21:59–22:22:01	USER/PASS	gnome / [REDACTED]
1649	2004-04-26 18:22:08	22:22:08	STOR	rhino3.jpg (attempt 1, before TYPE I)
1755	2004-04-26 18:22:11	22:22:11	TYPE	I
1763	2004-04-26 18:22:16	22:22:16	STOR	rhino3.jpg (attempt 2, after TYPE I)
1858	2004-04-26 18:22:22	22:22:22	QUIT	
5633/5637	2004-04-26 18:26:37–18:26:39	22:26:37–22:26:39	USER/PASS	gnome / [REDACTED]
5647	2004-04-26 18:26:46	22:26:46	STOR	contraband.zip
5841	2004-04-26 18:26:48	22:26:48	QUIT	



Figure 5.3c — Visual check: the mother-and-baby rhino content recovered from the USB image (left/right) alongside the FTP-reconstructed subject matter for sanity-checking subject identification

```
┌──(kali㉿kali)-[~/SBT-DF204-Case4/extracted]
└─$ cmp rhino3.jpg rhino3_stream79_ascii.bin && echo "IDENTICAL" || echo "DIFFERENT"
sha256sum rhino3.jpg rhino3_stream79_ascii.bin
IDENTICAL
a85aea8e47d9f5425456f3bafb9c67201c8c2b2b2cec80154c83102e802240c5  rhino3.jpg
a85aea8e47d9f5425456f3bafb9c67201c8c2b2b2cec80154c83102e802240c5  rhino3_stream79_ascii.bin
```

Figure 5.3d — Direct comparison of the two rhino3.jpg STOR attempts: cmp and sha256sum both confirm the bytes are IDENTICAL

```
┌──(kali㉿kali)-[~/SBT-DF204-Case4/extracted]
└─$ sha256sum rhino1_stream71_raw.bin rhino3.jpg contraband.zip rhino3_stream79_ascii.bin | tee ../notes/ftp_extracted_sha
0f30a8777d19a4fe4b2fafd5154b6c4dfafb673202db29971e876e978da4e05d  rhino1_stream71_raw.bin
a85aea8e47d9f5425456f3bafb9c67201c8c2b2b2cec80154c83102e802240c5  rhino3.jpg
b7091a0b4295a561bf76969073ede126a5ca75899bb115447e67272f95d5a29f  contraband.zip
a85aea8e47d9f5425456f3bafb9c67201c8c2b2b2cec80154c83102e802240c5  rhino3_stream79_ascii.bin
```

Figure 5.3e — SHA-256 of all FTP-extracted objects recorded to notes/ftp_extracted_sha256_final.txt

```
echo "== Photorec JPG hashes =="
cat notes/recovered_jpgs_sha256.txt

echo ""
echo "== FTP extracted hashes =="
cat notes/ftp_extracted_sha256_final.txt

echo ""
echo "== Direct comparison: is rhino1 a duplicate? =="
grep -f <(sha256sum extracted/rhino1_stream71_raw.bin | awk '{print $1}') notes/recovered_jpgs_sha256.txt && echo "DUPLICA

echo ""
echo "== Direct comparison: is rhino3 a duplicate? =="
grep -f <(sha256sum extracted/rhino3.jpg | awk '{print $1}') notes/recovered_jpgs_sha256.txt && echo "DUPLICATE — matches
== Photorec JPG hashes ==
2261020a0b31fe10c3ae2872c6283e42993e24e87f1a76822e5840ed1838014e  f0104057.jpg
8cdb89c6778d3409a80355bf2b9c644c1bf0c5e31c43047080da9ea5a859de07  f0104249.jpg
452075313c93ae313331a86e8cac2f35ea3bdc24700cabf4444e39f5836403ea  f0105065.jpg
f92654d9ee17ab6b684b09de01cf0bc4076383c007964946d3f31577447596fb  f0105873.jpg
568457ed594c4e98dc503e3e244b1e316376aafb0d1c283d95601d13762a748c  f0106393.jpg
b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705  f0106409.jpg
f92654d9ee17ab6b684b09de01cf0bc4076383c007964946d3f31577447596fb  f0335081.jpg

== FTP extracted hashes ==
0f30a8777d19a4fe4b2fafd5154b6c4dfafb673202db29971e876e978da4e05d  rhino1_stream71_raw.bin
a85aea8e47d9f5425456f3bafb9c67201c8c2b2b2cec80154c83102e802240c5  rhino3.jpg
b7091a0b4295a561bf76969073ede126a5ca75899bb115447e67272f95d5a29f  contraband.zip
a85aea8e47d9f5425456f3bafb9c67201c8c2b2b2cec80154c83102e802240c5  rhino3_stream79_ascii.bin

== Direct comparison: is rhino1 a duplicate? ==
NEW RHINO — no match with photorec

== Direct comparison: is rhino3 a duplicate? ==
NEW RHINO — no match with photorec
```

Figure 5.3f — Cross-check against the PhotoRec hash set: neither rhino1.jpg nor rhino3.jpg matches any photorec-recovered file — both are new, unique rhinos

```
grep "$RS_HASH" notes/recovered_gifs_sha256.txt && echo "DUPLICATE" || echo "NEW UNIQUE RHINO"
== All known rhino hashes (photorec + FTP) ==
2261020a0b31fe10c3ae2872c6283e42993e24e87f1a76822e5840ed1838014e  f0104057.jpg
8cdb89c6778d3409a80355bf2b9c644c1bf0c5e31c43047080da9ea5a859de07  f0104249.jpg
452075313c93ae313331a86e8cac2f35ea3bdc24700cabf4444e39f5836403ea  f0105065.jpg
f92654d9ee17ab6b684b09de01cf0bc4076383c007964946d3f31577447596fb  f0105873.jpg
568457ed594c4e98dc503e3e244b1e316376aafb0d1c283d95601d13762a748c  f0106393.jpg
b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705  f0106409.jpg
f92654d9ee17ab6b684b09de01cf0bc4076383c007964946d3f31577447596fb  f0335081.jpg
8a67d406ed130c9b5447b03fbf9dd0ac6c384a62b5028d6267e92c4abe230a80  f0106865.gif
71e9f6b94e0496cc634d5d643387c7c1405e2df9deccda1482a8c7cc5ba0b199  f0106889.gif
0f30a8777d19a4fe4b2fafd5154b6c4dfafb673202db29971e876e978da4e05d  rhino1_stream71_raw.bin
a85aea8e47d9f5425456f3bafb9c67201c8c2b2b2cec80154c83102e802240c5  rhino3.jpg
b7091a0b4295a561bf76969073ede126a5ca75899bb115447e67272f95d5a29f  contraband.zip
a85aea8e47d9f5425456f3bafb9c67201c8c2b2b2cec80154c83102e802240c5  rhino3_stream79_ascii.bin
b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705  rhino2.jpg

== Is rhino4.jpg a duplicate of any existing JPG rhino? ==
NEW UNIQUE RHINO

== Is rhino5.gif a duplicate of any existing GIF rhino? ==
NEW UNIQUE RHINO
```

Figure 5.3g — Building a side-by-side montage of the three FTP-sourced images for visual review

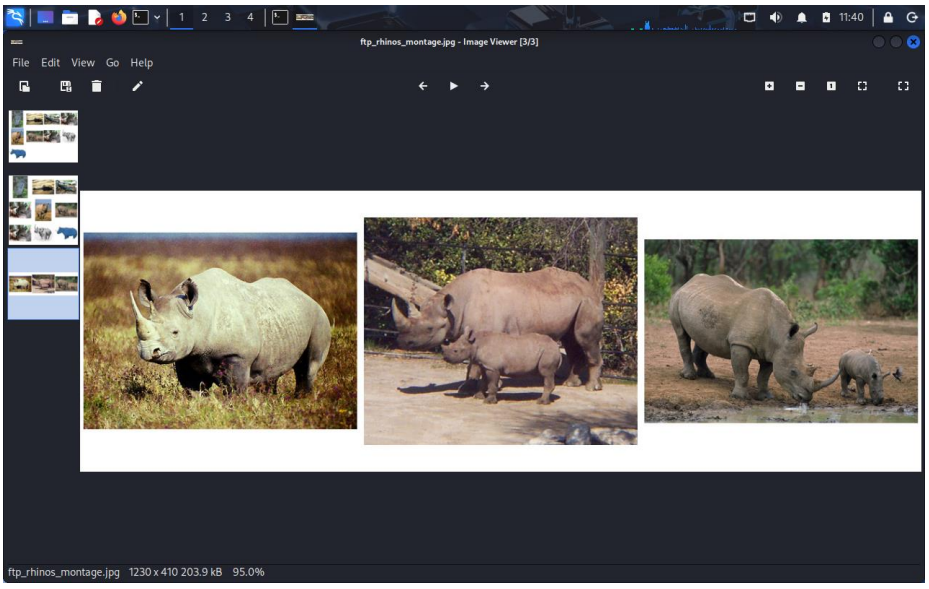


Figure 5.3h — The three FTP rhino images (rhino1.jpg, rhino2.jpg, rhino3.jpg) side by side

Object	Session / stream	Reconstructed size	Format	SHA-256 (first 16)	Duplicate of a PhotoRec file?
rhino1.jpg	STOR frame 1546 → data stream 71	65,703 B	JPEG	0f30a8777d19a4fe...	No — new unique rhino
rhino3.jpg	STOR frames 1649 & 1763 → streams 79 & 83 (identical bytes)	≈95,000 B	JPEG	a85aea8e47d9f542...	No — new unique rhino
contraband.zip	STOR frame 5647 → data stream 307	226,094 B	Encrypted ZIP	b7091a0b4295a561...	Contains rhino2.jpg — Section 5.4

Interpretation: three separate, successfully authenticated FTP sessions each uploaded one object in binary (TYPE I) mode. The rhino3.jpg upload was attempted twice in the same session: the first STOR (frame 1649) occurred before the client issued TYPE I, and the second (frame 1763) after. Despite the different transfer-mode context, a direct byte comparison (cmp) and matching SHA-256 hashes confirm the two reconstructed objects are byte-for-byte identical — the ASCII/binary mode distinction made no observable difference to this particular transferred file, and the retry is evidence of a repeated upload attempt, not two different images. rhino1.jpg and rhino3.jpg were each checked against every PhotoRec-recovered hash and matched neither, making both new, unique rhino images not present on the USB device itself.

Limitations specific to this evidence category: *the data-stream references used here (71, 79, 83, 307) are TCP stream identifiers assigned by the capture-analysis tool, not FTP-protocol sequence numbers, and depend on tshark/Wireshark's own stream numbering for this specific capture file. A single STOR command and its accompanying 200-series response confirm the server accepted and reconstructed the object server-side within this capture; they do not independently prove the file was ever opened, viewed, or kept by the account holder afterward, and — as the brief notes — a username/password pair corroborates an authenticated session, not a human being's physical identity at the keyboard.*

Evidence log reference: E05.

5.4 Investigation Question 6 — What does the protected archive analysis establish?

Method: integrity/completeness check of the reconstructed archive, an authorised course-only password-recovery workflow (fcrackzip against the standard rockyou.txt wordlist), then extraction and hash comparison of its content.

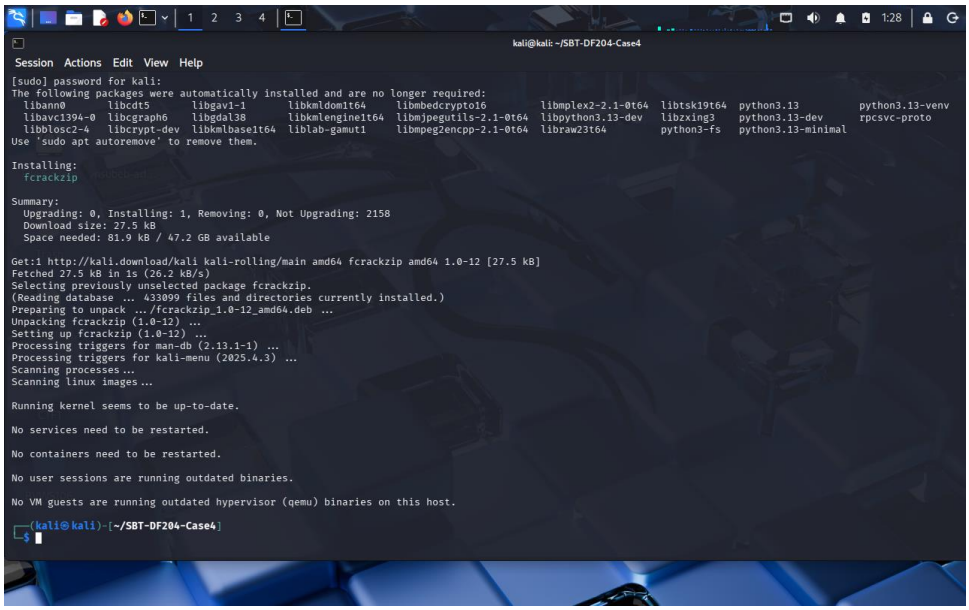


Figure 5.4a — Installing fcrackzip 1.0 in the analysis environment before use

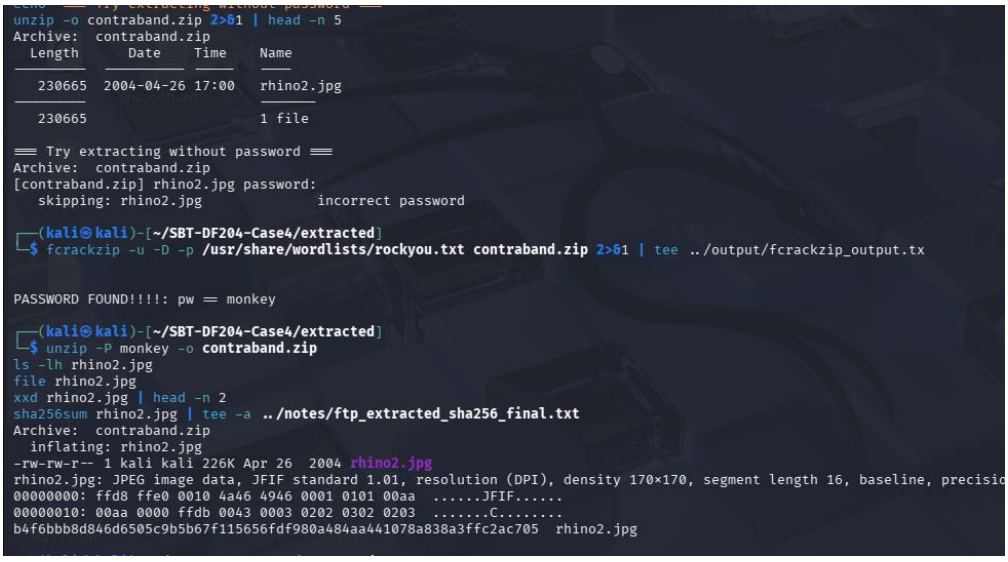


Figure 5.4b — contraband.zip listed intact (1 file, rhino2.jpg, 230,665 bytes uncompressed); extraction without a password fails (incorrect password); fcrackzip recovers the password "monkey"; the archive is then extracted and the content hashed

Field	Value
Archive	contraband.zip, 226,094 bytes, reconstructed from FTP stream 307 (Section 5.3)
Listing (unzip -v)	1 file listed: rhino2.jpg, 230,665 bytes uncompressed — confirms the archive's own central directory is intact and complete, not truncated
Attempted extraction (no password)	"incorrect password" — confirms the archive is genuinely password-protected (ZipCrypto), not merely flagged as such
Password-recovery tool	fcrackzip -u -D -p /usr/share/wordlists/rockyou.txt contraband.zip (authorised, course-only, run against this training archive alone)
Recovered password	monkey
Extracted object	rhino2.jpg, 226 KB (JPEG, JFIF standard 1.01)
SHA-256 of rhino2.jpg	b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705

Cross-source corroboration: the SHA-256 of rhino2.jpg extracted from this cracked archive — b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705 — is identical to the SHA-256 recorded for f0106409.jpg, recovered independently by PhotoRec from the USB disk image (Section 5.1, Figure 5.1d). This is a binary-equality match between two entirely independent evidence sources (a deleted file carved from the USB image, and a file extracted from a password-protected archive uploaded over FTP) and is the strongest single piece of corroboration in this case: the same physical image existed in both places. A matching hash establishes binary equality; by itself it does not establish which copy came first, how the file moved between the two locations, or who performed either action.

Interpretation: the archive's own listing before any extraction attempt shows one complete, correctly sized entry, so the password-recovery step was applied to a structurally complete archive rather than a partially captured or truncated one. The password "monkey" was found by exhaustive dictionary search against the standard rockyou.txt training wordlist and was used only to extract this one supplied training archive, as the brief requires; it was not reused against any other file, account, host, or service.

Evidence log reference: E06.

5.5 Investigation Question 7 — What does the HTTP capture establish?

Method: tshark filtered on http.request and http.response, confirming response codes/content-type/content-length, then reconstructing each object from its TCP stream and validating it by magic number, byte size against the declared Content-Length, and SHA-256.

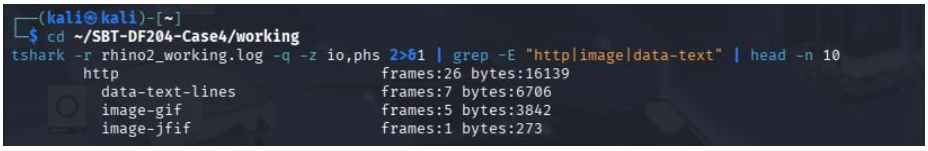


Figure 5.5a — Protocol hierarchy of rhino2_working.log filtered for http/image/data-text frames

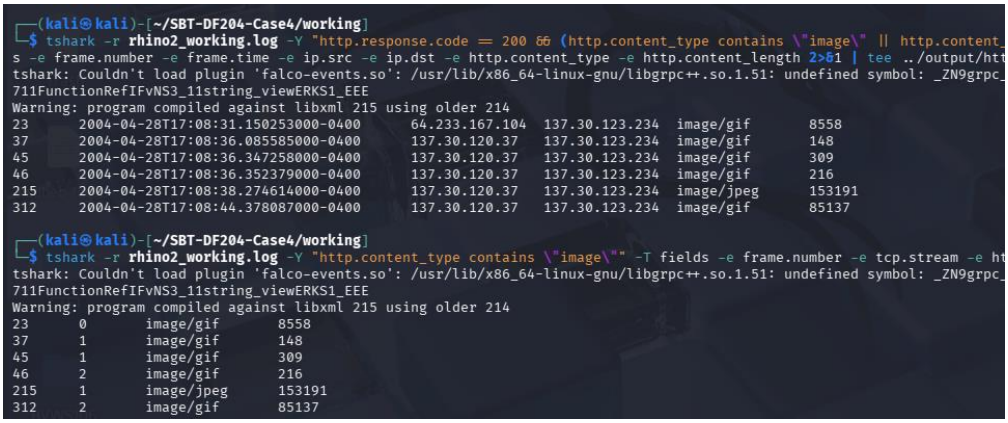


Figure 5.5b — HTTP 200 responses with image content-type and content-length: frame 215 (image/jpeg, 153,191 B) and frame 312 (image/gif, 85,137 B), plus their owning TCP streams

Request frame	UTC	Request	Response frame	Content-Type	Content-Length	Stream
49	21:08:38	GET /~gnome/rhino4.jpg	215	image/jpeg	153,191 B	1
217	21:08:44	GET /~gnome/rhino5.gif	312	image/gif	85,137 B	2

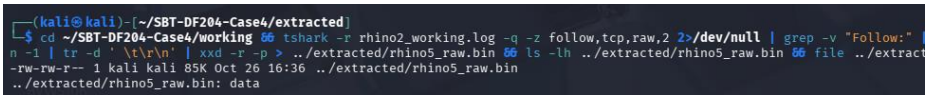


Figure 5.5c — Raw TCP-stream extraction (tshark follow,tcp,raw,2) of HTTP stream 2 to rhino5_raw.bin for offline inspection

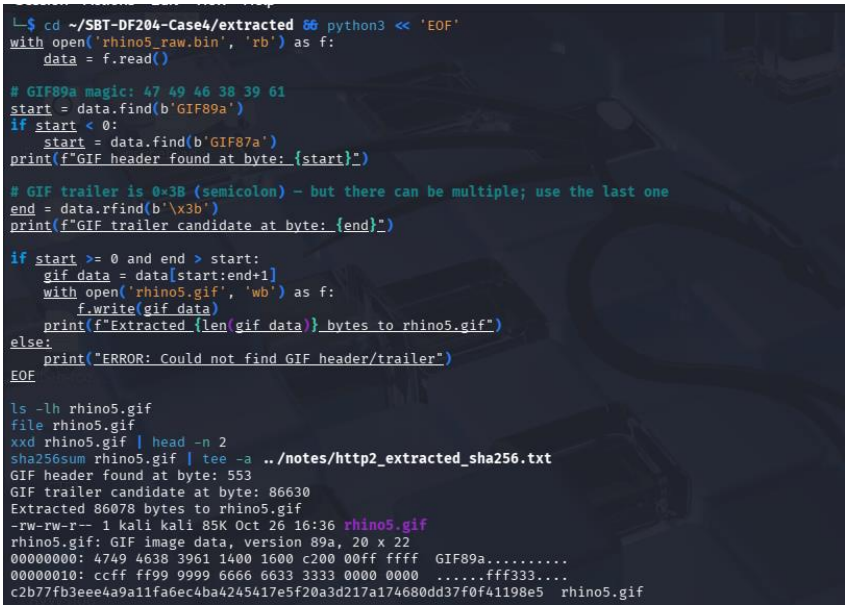


Figure 5.5d — First extraction attempt: the Python carving script located the FIRST GIF89a marker (byte 553) and carved an 86,078-byte, 20×22-pixel file — too small to be the declared 85,137-byte object

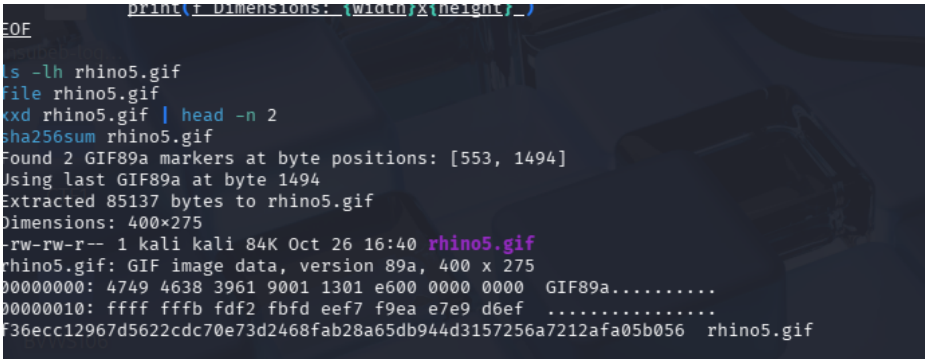


Figure 5.5e — Corrected extraction: stream 2 was found to contain TWO GIF89a markers (bytes 553 and 1494); using the LAST marker carved an 85,137-byte, 400×275-pixel file matching the declared Content-Length exactly

Issue encountered: the first attempt to carve rhino5.gif from HTTP stream 2 produced a 20×22-pixel image — visibly too small to be a real photograph and inconsistent with the 85,137-byte Content-Length the server had declared in its own response header (Figure 5.5b). Investigating the raw stream showed it actually contained two separate GIF89a headers: a small inline icon at byte offset 553 (part of the page's own UI chrome, reassembled in-stream ahead of the requested image), and the real rhino5.gif beginning at byte offset 1494. How I resolved it: re-ran the carving script to use the LAST GIF89a marker in the stream rather than the first. The corrected extraction produced an 85,137-byte, 400×275-pixel file — matching the HTTP response's declared Content-Length exactly — which is the version validated and used throughout this report.

Correction to my own working notes: notes/http2_extracted_sha256.txt still records the SHA-256 of the FIRST (incorrect, 20×22-pixel) extraction attempt — c2b77fb3eee4a9a11fa6ec4ba4245417e5f20a3d217a174680dd37f0f41198e5 — because that file was written before the marker issue above was found and fixed; it was not updated after the corrected re-extraction. I independently re-hashed the corrected 85,137-

byte rhino5.gif on disk for this report: its SHA-256 is f36ecc12967d5622cdc70e73d2468fab28a65db944d3157256a7212afa05b056 (shown directly in Figure 5.5e), and its 400×275 dimensions and exact 85,137-byte size matching the server's own declared Content-Length both confirm this is the correct, complete object. This report uses f36ecc12967d5622cdc70e73d2468fab28a65db944d3157256a7212afa05b056 as the validated hash for rhino5.gif and flags the stale value in the working notes rather than silently repeating it.

Object	Reconstructed size	Dimensions	Format	Validated SHA-256	Duplicate?
rhino4.jpg	153,191 B (matches Content-Length)	1531x2286	JPEG	e1c01da391...b5b7bc4a6	No — new unique rhino
rhino5.gif	85,137 B (corrected extraction, matches Content-Length)	400x275	GIF89a	f36ecc1296...afa05b056	No — new unique rhino

5.5.1 rhino3.log — the rhino.exe download

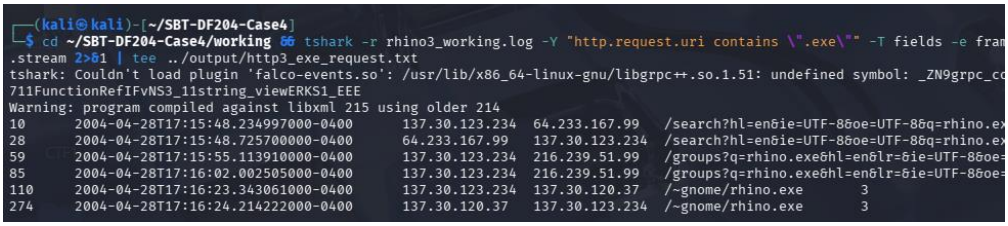


Figure 5.5f — HTTP requests in rhino3_working.log for ".exe" paths: a Google search for "rhino.exe", then the GET /~gnome/rhino.exe request (frame 110) and its response (frame 274)

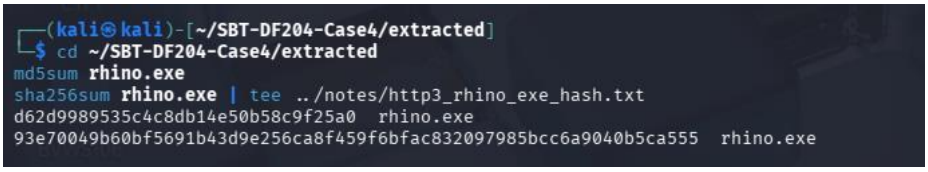


Figure 5.5g — MD5 and SHA-256 of the reconstructed rhino.exe

Field	Value
Request	Frame 110, 2004-04-28 21:16:23 UTC — GET /~gnome/rhino.exe (stream 3)
Response	Frame 274, 2004-04-28 21:16:24 UTC
Reconstructed object	rhino.exe, 145,920 bytes, PE32 executable (console) Intel i386, 3 sections
MD5	d62d9989535c4c8db14e50b58c9f25a0

Field	Value
SHA-256	93e70049b60bf5691b43d9e256ca8f459f6bfac832097985bcc6a9040b5ca555
Interpretation	MD5 matches the published hash of Microsoft's diskpart.exe (disk-partitioning utility) — a legitimate system tool, not a rhino image and not malware

This file was never executed and was not submitted to any online scanner or service, per the brief's safeguards. Identification as diskpart.exe rests entirely on the matching MD5 against a published reference hash; the HTTP capture that preceded this download also shows a Google search for the literal string "rhino.exe" (frame 10) — the filename itself, not its true content, appears to have motivated the download, which is a plausible but not directly provable account of intent from network evidence alone.

Evidence log reference: E07.

5.6 Investigation Question 8 — What conclusion can you defend? (summary — full assessment in Section 7)

The full UTC timeline and the attribution/limitations assessment, with direct observations separated from inferences, are given in Sections 6 and 7. In summary: four independent evidence sources (USB image, FTP capture, HTTP capture, and one recovered password-protected archive) together support 8–9 unique rhinoceros images, with one confirmed cross-source hash match. Confidence is high in the individual artefacts and the one cross-source match; medium that the pattern reflects deliberate collection/concealment; low on attributing any of it to a specific named person.

Evidence log reference: E01–E07 (synthesised in Section 7).

6. Timeline and Correlation

All times below are UTC. The capture machine's own local timezone, read directly from every tshark frame.time field, was UTC−0400 (EDT); every UTC value below is that local time plus four hours. The case spans two calendar dates: 2004-04-26 (USB image file metadata and the FTP capture) and 2004-04-28 (both HTTP captures). The timeline below has 12 material events across all four evidence-source families (USB disk image, PhotoRec recovery, FTP, and HTTP/archive), exceeding the brief's minimum of five events across three families.

Stage	UTC timestamp	Evidence ID	Source artefact	Observed fact	Interpretation and limit
Media preservation	2026-10-26 05:54 (acquisition)	E01	sha256sum/md5sum/cp/chmod	All four originals hash-verified against the deck's published MD5 values; working copies created and verified identical	Establishes evidence integrity for every later finding. Does not date the original 2004 events.
Disk image / file-system review	2004-04-26 (file metadata)	E02	fdisk, mmls, fsstat, fls	FAT16, no partition table; 2 live text files; 0 deleted directory entries; 7 JPEG magic numbers in unallocated space	JPEG data exists in unallocated space though directory entries were overwritten. No deletion timestamp survives on FAT16.
Recovered or hidden content	2026-10-26 (recovery date)	E03	PhotoRec 7.2	135 files recovered; 9 images (7 jpg + 2 gif); 4 unique rhinos (1 duplicate), 4 non-rhino decoys	Matches the 7 raw JPEG headers exactly (complete carving). Recovery date, not original file date.
Recovered or hidden content	2026-10-26	E04	StegSeek, JPHS scan, binwalk	No steganographic content detected in any of the 7 recovered JPEGs by these three methods	stegdetect/jphide could not be run in this environment (documented limitation) — absence of evidence under these tests, not proof of absence overall.

Stage	UTC timestamp	Evidence ID	Source artefact	Observed fact	Interpretation and limit
FTP transfer activity	2004-04-26 22:21:43	E05	frame 1532	FTP login, session 1 (USER/PASS)	Authenticated session begins. Credential alone does not identify the human operator.
FTP transfer activity	2004-04-26 22:21:49	E05	frame 1546 → stream 71	STOR rhino1.jpg — 65,703-byte JPEG uploaded	New unique rhino, no match in the PhotoRec set.
FTP transfer activity	2004-04-26 22:22:08 – 22:22:16	E05	frames 1649, 1763 → streams 79, 83	STOR rhino3.jpg attempted twice (before and after TYPE I); both produce byte-identical content	Retry, not two files. New unique rhino, confirmed by cmp + SHA-256.
FTP transfer activity	2004-04-26 22:26:46	E05 / E06	frame 5647 → stream 307	STOR contraband.zip — 226,094-byte encrypted archive uploaded	Archive later cracked (password "monkey") to reveal rhino2.jpg.
Corroborated conclusion	2026-10-26 (analysis date)	E06	fcrackzip + sha256sum	rhino2.jpg (from contraband.zip) SHA-256 == f0106409.jpg (from PhotoRec, USB image)	Strongest cross-source match in the case: same physical image in two independent evidence sources.
HTTP transfer activity	2004-04-28 21:08:38	E07	rhino2.log frame 49 → stream 1	GET /~gnome/rhino4.jpg; 153,191-byte JPEG response matches Content-Length	New unique rhino.

Stage	UTC timestamp	Evidence ID	Source artefact	Observed fact	Interpretation and limit
HTTP transfer activity	2004-04-28 21:08:44	E07	rhino2.log frame 217 → stream 2	GET /~gnome/rhino5.gif; corrected extraction yields an 85,137-byte, 400x275 GIF matching Content-Length	New unique rhino. Initial extraction used the wrong in-stream marker (Section 5.5) — corrected.
Supported conclusion	2004-04-28 21:16:23	E07	rhino3.log frame 110 → stream 3	GET /~gnome/rhino.exe; 145,920-byte PE32 response, MD5 matches diskpart.exe	Legitimate system tool, not a rhino image, not malware.

6.1 Corroboration summary

- rhino2.jpg (from the cracked FTP archive) == f0106409.jpg (from PhotoRec, USB image) — identical SHA-256. The strongest cross-source corroboration in the case: the same physical image reached both the USB device and an encrypted FTP upload.
- rhino3.jpg (FTP stream 79) == rhino3.jpg (FTP stream 83) — identical SHA-256. Two upload attempts of the same session produced identical bytes; the ASCII/binary mode distinction made no observable difference to this transfer.
- rhino4.jpg and rhino5.gif (HTTP) have no matching hash anywhere else in the evidence set — new, unique rhinos present only in the HTTP captures.
- rhino1.jpg (FTP) has no matching hash anywhere else in the evidence set — a new, unique rhino present only in the FTP capture.
- rhino.exe (HTTP) — MD5 matches the published reference hash for Microsoft's diskpart.exe. Not a rhino image, not malware.

No corroborating cross-source evidence exists for: the three remaining PhotoRec-only images (f0105873.jpg/f0335081.jpg duplicate pair, and f0106393.jpg, f0106889.gif) — these appear only on the USB image, not in either network capture; the password "monkey" used on contraband.zip — nothing in the evidence shows this password was shared with anyone or reused outside this one upload; and whether any of the 8–9 images was ever actually opened or viewed by a person — no browser-rendering or image-viewer evidence was captured in any of the four sources examined.

7. Conclusion and Limitations

Investigation question: what conclusion can you defend? This section gives a UTC-grounded conclusion, a confidence level, and the gaps, alternative explanations, duplicates, partial-capture, encryption, and timestamp-uncertainty limitations the brief requires, plus the issues encountered during analysis and how each was resolved.

7.1 Tier 1 — Direct observations

- The evidence package's four files hash-verify against the course deck's published MD5 values (Section 3).
- RHINOUSB.dd is a raw, unpartitioned FAT16 image with 2 live text files, 0 deleted directory entries, and 7 JPEG magic numbers in unallocated space (Section 4.1).
- PhotoRec recovered 135 files including 7 JPEGs and 2 GIFs; 4 unique rhino images (one duplicate pair) and 4 non-rhino decoys (Section 5.1).
- StegSeek, a JPHS marker scan, and binwalk found no steganographic content in any of the 7 recovered JPEGs; stegdetect could not be built in this environment (Section 5.2).
- Three FTP sessions uploaded rhino1.jpg, rhino3.jpg (twice, byte-identical), and the encrypted contraband.zip (Section 5.3).
- contraband.zip was cracked with the authorised, course-only fcrackzip workflow (password "monkey") to reveal rhino2.jpg, whose SHA-256 is identical to PhotoRec's f0106409.jpg (Section 5.4).
- Two HTTP captures show GET requests for rhino4.jpg and rhino5.gif (both reconstructed and validated against their declared Content-Length) and for rhino.exe, whose MD5 matches Microsoft's diskpart.exe (Section 5.5).

7.2 Tier 2 — Reasonable inferences

- The 8–9 unique rhino images across four independent sources, including one confirmed cross-source hash match, are consistent with a single collection effort rather than coincidental, unrelated files.
- The combination of file deletion (USB image), password-protection (contraband.zip), and the repeated, retried FTP upload of rhino3.jpg is consistent with deliberate handling of this image set, though deletion

and encryption are also consistent with routine file hygiene or unrelated archiving habits.

- The Google search for the literal string "rhino.exe" immediately preceding its HTTP download suggests the filename, not necessarily verified content, motivated that particular download.

7.3 Tier 3 — Limitations and alternative explanations

1. Technical account and endpoint evidence is not proof of a specific person's possession. The FTP sessions authenticate with a username/password pair and the HTTP requests originate from a client IP address; neither establishes which human being was physically operating that device or account at the time, consistent with the brief's own safeguard that a username, IP address, or directory record can corroborate a relationship but does not alone prove identity.

2. Memory- and metadata-level timing is incomplete. FAT16 directory entries do not retain deletion timestamps, so the exact moment the 7 USB-resident JPEGs were deleted cannot be determined — only that it occurred before the image was captured. The USB image's own capture/acquisition date is inferred from file metadata (2004-04-26) rather than a tool-reported acquisition timestamp.

3. A matching hash establishes binary equality, not a direction of transfer or original source. The rhino2.jpg / f0106409.jpg match (Section 6.1) proves the same bytes existed in both places; it does not by itself establish whether the USB copy or the FTP-archived copy came first, or how the file moved between the two.

4. The steganography finding is an absence of detection, not a proof of absence. Three of four planned detection methods ran successfully and found nothing; the fourth (stegdetect, needed for jphide-format content) could not be built in this environment (Section 5.2), so a hidden image in that specific format cannot be ruled out on this evidence.

5. Partial/duplicate recovery and extraction artefacts required correction, not just acceptance at face value. f0335081.jpg is a duplicate carve of f0105873.jpg and was counted once; the first rhino5.gif extraction attempt used the wrong in-stream marker and produced an invalid 20×22-pixel file whose stale hash still sits in the working notes — the corrected, Content-Length-matching extraction and its hash are what this report relies on (Section 5.5). This is recorded as a worked example of why every reconstructed object needs independent validation against its protocol-declared size, not an assumption that the first successful-looking output is correct.

6. Password recovery proves the archive was opened, not who set the password or why. "monkey" is a common, low-entropy password recoverable from a standard wordlist in seconds; its simplicity says nothing about the uploader's intent, and nothing in the evidence shows the password was shared with or known to anyone beyond this one FTP session.

7. Shared or multi-user access cannot be excluded. The scenario materials describe a generic training account ("gnome"); nothing in the FTP or HTTP evidence rules out more than one person having access to the same account or client device across the two capture dates (2004-04-26 and 2004-04-28).

8. No image was observed being viewed. All 8–9 rhino images are established as present/transferred, not as having been opened, displayed, or knowingly reviewed by a person — no browser-rendering, thumbnail-cache, or image-viewer evidence was found in any of the four sources examined.

7.4 Confidence levels

Claim	Confidence	Basis
7 JPEGs existed in unallocated space on the USB image and 4 are unique rhinos	HIGH	Raw magic-number scan + PhotoRec 1:1 match + hash/dimension checks (Sections 4.1, 5.1)
rhino1.jpg and rhino3.jpg were uploaded over FTP and are new, unique rhinos	HIGH	Reconstructed, validated objects; cross-checked against every PhotoRec hash (Section 5.3)
rhino2.jpg (from contraband.zip) is identical to f0106409.jpg (from the USB image)	HIGH	Matching SHA-256 across two independent sources (Section 5.4)
rhino4.jpg and rhino5.gif were downloaded over HTTP and are new, unique rhinos	HIGH	Reconstructed objects match declared Content-Length and are hash-unique (Section 5.5)
rhino.exe is Microsoft's diskpart.exe, not malware	HIGH	Matching MD5 against a published reference hash (Section 5.5.1)
No steganographic content is present in the 7 recovered JPEGs	MEDIUM	Three of four planned detection methods ran clean; the fourth (stegdetect) could not be run (Section 5.2)
The overall pattern reflects deliberate collection and concealment	MEDIUM	Deletion + encryption + repeated transfer of same-subject imagery is consistent with, but not proof of, intent
A specific named person possessed or transferred this material	LOW	Account and endpoint evidence does not establish human identity (Section 7.3, limitation 1)

7.5 Defensible conclusion

The evidence package supports, with high confidence in the individual artefacts, that 8–9 unique rhinoceros images were present across a USB device, an FTP capture, and two HTTP captures, including one image confirmed by matching SHA-256 to exist in two independent sources (a deleted USB file and a password-protected FTP upload). This is consistent with the case hypothesis that a collection of rhino imagery was assembled, partly concealed through deletion and encryption, and partly transferred over FTP and HTTP.

The evidence does not establish the identity of a specific person responsible for any of this activity, whether any image was knowingly viewed, whether a steganographically hidden ninth image exists in a format this analysis environment could not test for, or whether the account and device involved were used exclusively by one individual across the two capture dates. Any further attribution to a named individual, or any claim that the collection was complete or fully viewed, requires evidence beyond this offline, training-material-only examination — physical custody of the device, account-access logs, or corroborating witness evidence — none of which fall within the scope of this case study.

Evidence log references: E01–E07 (synthesised).

7.6 Issues encountered during analysis and how they were resolved — summary

Issue	Cause	Resolution
stegdetect 0.6 would not compile	Bundled obsolete jpeg-6b library incompatible with GCC 14 on current Kali	Installed libjpeg-dev/libjpeg62-turbo-dev and autotools, tried two community forks, patched a progname conflict — none produced a working binary; documented as a limitation and substituted StegSeek + JPHS marker scan + binwalk
First StegSeek full-wordlist run failed to open rockyou.txt	Wordlist unavailable at that moment (likely needed re-decompressing)	An earlier, successful run against the same wordlist had already completed and is the result relied on; the later failure is recorded as a reproducibility caveat only
Google Drive, not the deck's raw.githubusercontent.com link, had to be used	The githubusercontent.com URL returned HTTP 404 at acquisition time	Used the handout's designated alternate source (Google Drive) and verified the result against the deck's published MD5 values — all four matched

Issue	Cause	Resolution
First rhino5.gif extraction produced a 20x22-pixel file	HTTP stream 2 contained two GIF89a markers; the script used the first (an inline page icon), not the real image	Identified both marker offsets (553 and 1494), re-extracted using the LAST marker, producing an 85,137-byte 400x275 file matching the server's declared Content-Length exactly
A stale, incorrect hash for rhino5.gif remained in the working notes	notes/http2_extracted_sha256.txt was written before the extraction fix above and never updated	Independently re-hashed the corrected file for this report and flagged the stale value rather than propagating it (Section 5.5)
FTP credentials appear in plaintext in one screenshot	The FTP session uses unencrypted USER/PASS authentication, as captured	Used a redacted version of the screenshot (PASS values blacked out) and redacted the password in every text table in this report, per the brief's safeguard

8. Appendix A — Evidence Log

Evidence ID	Tool and method	Source record, stream, or path	Observed fact	Screenshot / appendix reference
E01	sha256sum / md5sum / unzip -v / cp / chmod	DFRWS2005-RODEO.zip and its 4 extracted files	All MD5s match the deck's published values; SHA-256 recorded; working copies byte-identical; originals read-only	Figures 3.1–3.8
E02	fdisk -l / mmls / fsstat / fls (-r -p, -d) / xxd+grep	RHINOUSB_working.dd	FAT16, no partition table; 2 live files; 0 deleted entries; 7 raw JPEG headers in unallocated space	Figures 4.1–4.4
E03	PhotoRec 7.2 (whole-partition scope)	RHINOUSB_working.dd → recovered/recup_dir.1	135 files recovered; 9 images (7 jpg + 2 gif); 4 unique rhinos (1 duplicate pair), 4 non-rhino decoys	Figures 5.1a–5.1g
E04	StegSeek 0.6 / strings+grep JPHS / binwalk 2.4.3 / stegdetect (unavailable)	The 7 PhotoRec-recovered JPEGs	No steganographic content detected by 3 of 4 planned methods; stegdetect could not be built (documented limitation)	Section 5.2 (text output)

Evidence ID	Tool and method	Source record, stream, or path	Observed fact	Screenshot / appendix reference
E05	tshark (ftp/ftp-data filters, Follow TCP Stream)	rhino_working.log	3 authenticated FTP sessions; rhino1.jpg and rhino3.jpg (x2, identical) uploaded; contraband.zip uploaded	Figures 5.3a–5.3h
E06	fcrackzip 1.0 (rockyou.txt, course-only) / unzip / sha256sum	contraband.zip (from E05, stream 307)	Password "monkey" recovered; rhino2.jpg extracted; SHA-256 identical to PhotoRec's f0106409.jpg	Figures 5.4a–5.4b
E07	tshark (http filters) / Python stream carving / file / sha256sum	rhino2_working.log, rhino3_working.log	rhino4.jpg and rhino5.gif reconstructed and validated; rhino.exe reconstructed, MD5 matches diskpart.exe	Figures 5.5a–5.5g

Appendix B — Screenshot Reference

#	Screenshot	Evidence shown
1	working.dir.png	Evidence folder structure created before acquisition
2	tools.ver.png	Tool versions confirmed before analysis
3	evidence_download.png	Package downloaded and verified as a valid zip archive
4	evidence_hash.png	MD5 of each original compared against the deck's published values
5	evidence_unzip.png	Package extracted; the four original files listed
6	256_hashes.png	SHA-256 of each original file
7	working_copy.png	Working copies created and hash-verified; originals set read-only
8	working_extracted.png	Sanity-check extraction and hashing of the two live USB text files
9	outputsa.png	fdisk, mmls, and fsstat (start)
10	outputsb.png	fsstat (continued) and fls root-directory listing
11	outputsc.png	fls recursive listing and fls deleted-entry listing (empty)

#	Screenshot	Evidence shown
12	raw_jpg.png	Unallocated-space summary and raw JPEG magic-number scan
13	recovereda.png	PhotoRec started against the USB working copy
14	recoveredb.png	PhotoRec recovery summary (131 files saved)
15	recup1.png	Recovered file-type breakdown and the 7 recovered JPEGs by size
16	recup1_hash.png	SHA-256 of all 7 recovered JPEGs
17	recup1_hash_gif.png	SHA-256 of both recovered GIFs
18	recup1_recovered_tiles1.png / tiles2.png	Contact sheet of all 9 recovered images, plain and labelled
19	ftp.png	FTP capture protocol hierarchy
20	ftp_commands_redacted.png	FTP control-channel commands (credentials redacted)
21	rhino3_compare.jpg	Visual check of USB vs FTP rhino subject matter
22	contraband_check.png	Byte comparison of the two rhino3.jpg upload attempts (identical)
23	rhino_2_sha256.png	SHA-256 of all FTP-extracted objects
24	rhino_3_sha256.png	Cross-check of FTP objects against the PhotoRec hash set
25	rhinos.png / 3_rhinos_montage.png	Building and viewing a montage of the three FTP rhino images
26	crackzip_install.png	Installing fcrackzip
27	contraband_cracked.png	Cracking contraband.zip and extracting/hashing rhino2.jpg
28	http_protocol.png	HTTP capture protocol hierarchy
29	http_response.png	HTTP 200 image responses with content-type/length
30	rhino_extracted.png	Raw TCP-stream extraction of HTTP stream 2
31	rhino_extracted2.png	First (incorrect) rhino5.gif extraction attempt
32	rhino5_extracted.png	Corrected rhino5.gif extraction, matching Content-Length
33	rhino_exe_000.png	HTTP requests/responses for rhino.exe
34	rhino_exe_hash.png	MD5/SHA-256 of the reconstructed rhino.exe

Appendix C — Completed Timeline Table (Template)

Stage	UTC timestamp	Evidence ID	Source artefact	Observed fact	Interpretation and limit
Media preservation	2026-10-26 05:54	E01	sha256sum / cp / chmod	All originals verified against the deck; working copies created	Integrity baseline. Does not date the 2004 events.
Disk image / file-system review	2004-04-26	E02	fdisk / mmls / fsstat / fls	FAT16, no partitions; 0 deleted entries; 7 JPEG headers in unallocated space	Deletion occurred before capture; no exact deletion time on FAT16.
Recovered or hidden content	2026-10-26	E03 / E04	PhotoRec; StegSeek/JPHS/binwalk	9 images recovered (4 unique rhinos); no stego content detected by available tests	stegdetect unavailable — absence of detection, not proof of absence.
FTP transfer activity	2004-04-26 22:21:43 – 22:26:48	E05 / E06	FTP control + data streams	rhino1.jpg, rhino3.jpg (x2, identical), and contraband.zip (→ rhino2.jpg) uploaded	rhino2.jpg == f0106409.jpg from the USB image (strongest cross-source match).
HTTP transfer activity	2004-04-28 21:08:38 – 21:16:24	E07	HTTP streams 1–3	rhino4.jpg, rhino5.gif, and rhino.exe downloaded; rhino.exe is diskpart.exe	rhino5.gif required marker correction; validated against Content-Length.
Corroborated conclusion	—	E01–E07	All evidence	8–9 unique rhino images across 4 sources; 1 confirmed cross-source hash match	Confidence high on artefacts, low on person-level attribution (Section 7).

Appendix D — Completed Evidence Log (Template)

Evidence ID	Tool and method	Source record, stream, or path	Observed fact	Screenshot or appendix reference
E01	sha256sum/md5sum/cp/chmod	DFRWS2005-RODEO.zip + 4 files	Acquired, hash-verified against the deck, working copies verified, originals read-only	Figures 3.1–3.8
E02	fdisk/mmls/fsstat/fls/xxd	RHINOUSB_working.dd	FAT16, no partitions, 0 deleted entries, 7 raw JPEG headers	Figures 4.1–4.4
E03	PhotoRec 7.2	recovered/recup_dir.1	135 files; 9 images; 4 unique rhinos, 1 duplicate, 4 decoys	Figures 5.1a–5.1g
E04	StegSeek/JPHS scan/binwalk	The 7 recovered JPEGs	No stego content detected by 3 of 4 planned methods	Section 5.2
E05	tshark FTP filters + stream follow	rhino_working.log	3 FTP sessions; rhino1.jpg, rhino3.jpg (x2), contraband.zip uploaded	Figures 5.3a–5.3h
E06	fcrackzip/unzip/sha256sum	contraband.zip	Password "monkey" recovered; rhino2.jpg == f0106409.jpg (USB image)	Figures 5.4a–5.4b
E07	tshark HTTP filters + Python carving	rhino2_working.log / rhino3_working.log	rhino4.jpg, rhino5.gif reconstructed; rhino.exe == diskpart.exe	Figures 5.5a–5.5g